

The Effect of Character-Level Noise on Tokenization Robustness in Sentiment Analysis

Patrick Yao

University of California, Merced
pyao2@ucmerced.edu

Abstract

Many language models tokenize text into units at the word, subword, or character level. These tokens are the foundation of how language models process text given by a user. However, if text is introduced which is rarely encountered, misspelled, or otherwise contains character-level noise, it can negatively affect the performance of the model. Additionally, this noise in the input text can cause fragmented tokenization which results in an increase in the number of tokens produced. In this paper, I aim to show how character-level noise produces fragmented tokens leading to a degradation of performance in a sentiment analysis task. I apply character-level noise to the SST-2 dataset in the form of swapping, insertion, and deletion, then utilize the DistilBERT model to perform sentiment classification on the noisy text. Additionally, I utilize a BART-based spelling corrector to correct the noisy-input text to determine the effectiveness of text restoration as a pre-processing step. The experiment shows that the character-level noise does have a measurable effect on the accuracy and number of tokens generated. Furthermore, it shows that the restoration of the noisy text has a considerable effect on the accuracy of the model especially at the highest perturbation severity.

1 Introduction

Currently, many large language models (LLMs) use a pre-processing method known as tokenization. Tokenization converts text into smaller units known as tokens in the form of words, subwords, or characters (Wang et al., 2024; Chai et al., 2024). These tokens form the basis for the input of LLMs, making them an important factor in determining the performance of a model (Chai et al., 2024; Pawar et al., 2025; Kojima et al., 2025). One example of subword tokenization is known as Byte-Pair Encoding (BPE) (Sennrich et al., 2016). BPE is a widely used tokenization approach employed by

many modern LLMs including ChatGPT. BPE constructs a vocabulary by merging the most frequent adjacent pairs. This allows rare or unseen words to be represented by decomposing them into subword pieces according to the learned merge rules (Sennrich et al., 2016; Provilkov et al., 2020). One of the main problems of BPE and tokenization in general is known as the "curse of tokenization" (Chai et al., 2024) where the LLM experiences increased sensitivity to character-level perturbations such as typographic errors. While simple for humans to understand, character-level noise on text can have an adverse effect on an LLM's performance and understanding, causing problems beginning in the tokenization step (Chai et al., 2024; Kojima et al., 2025; Moradi and Samwald, 2021; Wang et al., 2024). In this paper, I attempt to answer the following questions:

1. Does character-level noise cause increased token fragmentation?
2. Does character-level noise and the increased token fragmentation lead to lower accuracy in sentiment analysis?
3. Can a pre-processing step via a BART-based spell correction model (Guhr et al., 2020) recover the accuracy lost by input perturbations?

To answer these questions, I employ a fine-tuned DistilBERT-based sentiment analysis model (Sanh et al., 2019) to conduct tests of accuracy and token fragmentation ratio on the clean, noisy, and corrected SST-2 datasets¹ (Socher et al., 2013).

2 Related Work

2.1 Tokenization Fragmentation and BPE

Tokenization fragmentation occurs when a rare or unseen word is broken down by a tokenization tech-

¹Code available at: <https://colab.research.google.com/drive/1o9yZIXfLyKiMo5g071F8FwZC2a9plyMF?usp=sharing>

nique such as Byte-Pair Encoding (BPE) (Sennrich et al., 2016). BPE is a process where a subword vocabulary is formed by merging the most frequent adjacent pairs in a corpus. During inference-time, when a word not previously seen in training is encountered, it is decomposed into subword pieces according to the learned merge rules. Although this process allows the LLM to handle unseen inputs, it is not a perfect process and subtle differences in token fragmentation or merge order can have a notable effect on the performance and output of the model (Pawar et al., 2025). Such adverse effects include sensitivity to inputs such as typographical errors, varying input lengths, and lack of awareness to its internal token structure. This sensitivity to inputs, which has been dubbed the "curse of tokenization" by Chai et al. (Chai et al., 2024), who find that even state-of-the-art (SOTA) models using BPE struggle against noisy inputs. Wang et al. (Wang et al., 2024) further demonstrate the effects of bad tokenization by constructing Adversarial Dataset for Tokenizer (ADT) to challenge various LLM tokenizers. Pawar et al. (Pawar et al., 2025) also find that the tokenization fragmentation process itself may induce a morphologically incorrect split of subwords which they propose can be fixed by including tokenization penalty. These small variations in tokenization can lead to enormous changes in the underlying understanding and overall output of a language model.

2.2 Character-Level Noise Robustness

Character-level noise such as typographical errors can have a substantial effect on the reasoning robustness of even SOTA LLMs. Although the noise may seem to be innocuous and inconsequential, these errors in the original input can have a compounding effect on the overall output (Gan et al., 2024; Kojima et al., 2025; Wang et al., 2024). To test the effects of character-level noise on reasoning robustness of LLMs, Gan et al. (Gan et al., 2024) create the Adversarial Typo Attack (ATA) algorithm which makes purposeful typographical errors in the input and determines the highest loss at each iteration. They find that the character-level noise created by ATA has a considerable effect on LLMs ranging from 5.6% to 13.6% reduction in response accuracy and performance across multiple datasets and models. Moreover, Kojima et al. (Kojima et al., 2025) similarly find inaccuracies ranging from 4.4% to 13.6% on sentiment analysis of the SST-2 dataset with added character-level

noise. These findings show that the robustness of a language model is meaningfully affected by character-level perturbations especially in the form of typographical errors seen in many real-world user inputs.

2.3 Benchmarking using Sentiment Analysis

Sentiment analysis is an area of Natural Language Processing (NLP) that tries to determine people's sentiments, emotions, and opinions through automated analysis of textual data. Sentiment analysis is an optimal candidate for benchmarking on account of it having clear labels for comparison while also requiring a complex understanding of the entire text to make an accurate prediction of its sentiment. Zhang et al. (Zhang et al., 2024) find that LLMs are suitable for tasks involving sentiment analysis and that on simple sentiment analysis tasks, they perform on par with small language models trained on specific sentiment analysis datasets such as SST-2 (Socher et al., 2013). Furthermore, they determine that on complex sentiment analysis tasks, LLMs are sensitive to inputs where a change in the prompt design can affect the general performance of the model. Given these findings, using the SST-2 dataset is an appropriate benchmark for testing sentiment analysis of perturbed inputs.

2.4 Mitigation

Mitigation of the effects of character-level noise on LLMs can be broken down into two different methods: training-time and inference-time mitigation techniques. An example of a training-time mitigation technique is BPE-Dropout (Provilkov et al., 2020) which randomly drops merges at each merge step. BPE-Dropout exposes the model to a wider variety of subword segmentation leading to a more robust model overall. However, although BPE-Dropout is an effective mitigation technique for noisy inputs, it has two notable drawbacks. Firstly, BPE-Dropout cannot be implemented on closed-source models as it needs to be implemented at training-time and access to the internal code of the model is required. Secondly, BPE-Dropout works only if BPE is the model's chosen tokenization method. If the model uses a tokenization method such as WordPiece namely BERT-based models like DistilBERT (Sanh et al., 2019), BPE-Dropout cannot be implemented without first modifying the model to work with BPE instead. Alternatively, an inference-time technique in the form of a spelling

correction model developed by Guhr et al. (Guhr et al., 2020), utilized in my study, can be used on both open- and closed-source models as well as models that do not utilize BPE.

3 Methodology

3.1 Task and Dataset

I utilize binary sentiment classification to determine if the overall sentiment of a given input is "positive" or "negative". To accomplish this, I employ the SST-2 dataset (Socher et al., 2013) from the GLUE benchmark to provide a set of labeled sentiment analysis examples to accurately determine performance. The validation split of this dataset was used which contains 872 labeled examples providing a sufficiently large sample size for reliable performance evaluation. Furthermore, the validation split was chosen over the test split due to each example containing exact positive and negative labels compared to the unlabeled test split. The validation split is also roughly balanced comprised of 50.9% positive examples and 49.1% negative examples.

3.2 Model

For this experiment, I chose DistilBERT’s "distilbert-base-uncased-finetuned-sst-2-english" model (Sanh et al., 2019) which has been fine-tuned for the SST-2 dataset (Socher et al., 2013). This specific model was chosen for its lightweight architecture, reasonable inference time, well-documented behavior, and its fine-tuning on the specific dataset used in this experiment. The model was accessed via HuggingFace’s pipeline for sentiment analysis and was used without any modification or retraining, ensuring that variations in the results of input perturbation tests were not affected by the model itself. Although DistilBERT employs WordPiece rather than BPE, both methods share the same subword fragmentation behavior when encountering rare or unseen words, making DistilBERT an appropriate model for studying the effects of token fragmentation. Moreover, a larger model was not chosen due to being too computationally expensive which was beyond the scope of this project.

3.3 Noise Injection

Noise is injected into the dataset via a custom scrambler function that applies character-level perturbations to each example. The function accepts

three inputs: the sentence to be perturbed, the severity level referred to as "chance", and a noise type. The noise type parameter specifies which of the three perturbation operations will be applied. The swap operation randomly swaps a selected character with the character at the following index within any word longer than two characters. The insert operation places a randomly selected character from the alphabet at a random valid position within a word. The delete operation removes a randomly selected character from a word. The chance parameter controls the per-word probability of perturbation and ranges from 0.0, where no words are modified, to 1.0, where every eligible word is perturbed, in increments of 0.1. The noise types follow the perturbation techniques established by Kojima et al. (Kojima et al., 2025) and Chai et al. (Chai et al., 2024) who have shown their validity as useful simulations of real-world typographical errors.

3.4 Token Fragmentation

To quantify the effect of character-level noise on the tokenization process, a Token Fragmentation Ratio (TFR) is computed at each severity level and for each noise type, measuring the degree to which noisy input increases token count relative to the clean baseline. TFR is formally defined as the ratio of the mean number of tokens produced by the perturbed dataset to the mean number of tokens produced by the clean baseline dataset.

$$\text{Token Fragmentation Ratio}(s) = \frac{\bar{T}_{\text{noisy}}(s)}{\bar{T}_{\text{clean}}} \quad (1)$$

where $\bar{T}_{\text{noisy}}(s)$ is the mean number of tokens produced by the tokenizer across all sentences at severity level s , and $\bar{T}_{\text{clean}}$ is the mean token count on the unperturbed clean text. A value of $\text{Token Fragmentation Ratio}(s) > 1$ indicates that noise at severity s causes increased tokenization fragmentation relative to the clean baseline.

3.5 Spell Correction

As an inference-time mitigation technique, the spelling correction model "oliverguhr/spelling-correction-english-base" by Guhr et al. (Guhr et al., 2020) is used as a pre-processing step on the perturbed text at each severity level before it is passed to the sentiment analysis model. The corrected text’s accuracy is then re-evaluated and compared against both the clean baseline and the uncorrected perturbed results. This approach requires no model

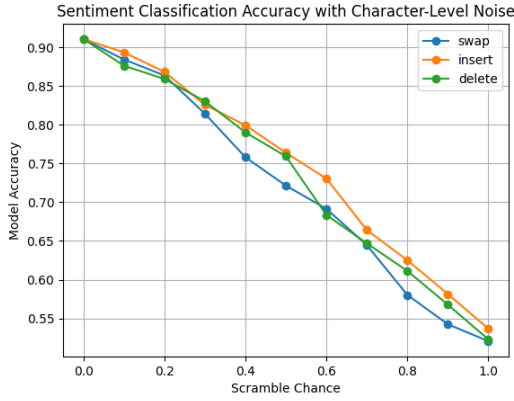


Figure 1: Sentiment classification accuracy of perturbed SST-2 dataset across eleven severity levels and three noise types.

retraining, as the correction occurs prior to the sentiment analysis model’s tokenization step.

4 Experiment

4.1 Baseline

The clean baseline accuracy for the unperturbed dataset was measured at 91.06%. This result serves as the comparison point for all the subsequent accuracy comparisons across perturbation severity levels and noise types.

4.2 Accuracy Under Noise

Figure 1 shows the accuracy of the DistilBERT sentiment analysis model (Sanh et al., 2019) on the SST-2 dataset (Socher et al., 2013) at each severity level and noise type. All three noise types exhibit a negative and roughly linear relationship between accuracy and severity level. They collectively experience a total decrease of roughly 39 percentage points from the baseline of 91.06%. Moreover, the three noise types ultimately converge towards a range of 52–54% accuracy at a severity of 1.0 suggesting that at the maximum perturbation level, noise type matters less than the amount of words perturbed. However, there are marginal but noticeable differences in the mid-range performance of each noise type. Particularly, the insert noise type maintains a marginally better accuracy than swap, with a maximum difference of 4.13% at a severity of 0.4. The relationship between accuracy drops and token fragmentation is explored in the following subsection.

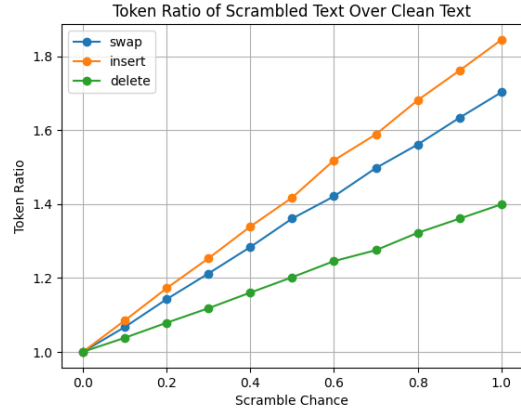


Figure 2: Mean ratio of tokens generated by the tokenizer between the noisy and clean datasets at each severity level for each noise type.

4.3 Token Fragmentation Ratio

As shown in Figure 2, all three noise types exhibit a positive and roughly linear relationship between severity and token fragmentation ratio, confirming that character-level noise consistently increases tokenization fragmentation relative to the clean baseline. Notably, the delete noise type shows considerably lower fragmentation than the other two types across all severity levels, which is consistent with the fact that deletion reduces the total number of characters in a word, producing shorter strings and less tokenization overall. Conversely, insert produces marginally higher fragmentation than swap, with the gap widening at higher severity levels, further supporting the observation that character count drives token count. Comparing these results to the accuracy degradation seen in Figure 1, the overall increase in token ratio is consistent with the general decrease in accuracy across all noise types. However, the marginal differences in accuracy between noise types are not fully accounted for by their considerable differences in token fragmentation ratio.

4.4 Summary Table

Table 1 shows the accuracy and Token Fragmentation Ratio (TFR) at each severity level for all three noise types. As severity increases, accuracy decreases and TFR increases across all noise types. The three noise types all start at 91.06% accuracy and a TFR of 1.000 at severity 0.0. At severity 1.0, accuracy converges to a range of 52.06–53.67% across all three noise types while TFR ranges more widely from 1.399 for delete to 1.844 for insert, suggesting that token fragmentation alone does not fully predict accuracy at higher severity levels.

Noise Type	Severity	Accuracy	TFR
Swap	0.0	91.06%	1.000
	0.1	88.42%	1.067
	0.2	86.35%	1.142
	0.3	81.42%	1.212
	0.4	75.80%	1.284
	0.5	72.13%	1.361
	0.6	69.15%	1.421
	0.7	64.45%	1.497
	0.8	58.03%	1.561
	0.9	54.24%	1.634
	1.0	52.06%	1.702
Insert	0.0	91.06%	1.000
	0.1	89.33%	1.085
	0.2	86.81%	1.172
	0.3	82.57%	1.253
	0.4	79.93%	1.339
	0.5	76.38%	1.417
	0.6	73.05%	1.518
	0.7	66.40%	1.588
	0.8	62.50%	1.681
	0.9	58.14%	1.761
	1.0	53.67%	1.844
Delete	0.0	91.06%	1.000
	0.1	87.61%	1.038
	0.2	85.89%	1.079
	0.3	83.03%	1.118
	0.4	79.01%	1.160
	0.5	75.92%	1.202
	0.6	68.35%	1.246
	0.7	64.68%	1.275
	0.8	61.12%	1.322
	0.9	56.77%	1.360
	1.0	52.29%	1.399

Table 1: Sentiment classification accuracy and Token Fragmentation Ratio (TFR) under character-level noise across severity levels.

5 Analysis

5.1 Relationship Between Fragmentation and Accuracy Loss

Based on the results from Figure 1 and Figure 2, there is a connection between the amount of token fragmentation and loss in accuracy. As TFR increases roughly linearly with severity, accuracy decreases at a comparable rate across all noise types. Additionally, the linear association between TFR and accuracy degradation holds for all noise types, demonstrating that noise type does not affect this relationship. However, the delete noise type produces the lowest TFR across all severity levels but does not consistently maintain the highest accuracy, suggesting that TFR alone does not fully account for accuracy differences between noise types.

5.2 Noise Type Comparison

Although all three noise types cause a roughly linear decrease in accuracy, their effects on model performance and token fragmentation differ. In Figure 1, insert maintains the highest accuracy at nearly every severity level, with the sole exception of the severity level of 0.3, where delete achieves a marginally higher accuracy than insert. While the three noise types remain close at lower severities, they gradually diverge in the mid-severity range, then converge again as they approach severity level 1.0. Moreover, in Figure 2, delete consistently shows a considerably lower token fragmentation ratio compared to swap and insert across all severity levels, while insert consistently produces the highest fragmentation. This suggests that the type of perturbation influences the degree of token fragmentation that occurs, yet regardless of noise type, all three cause a steady and consistent decrease in model performance.

5.3 Analysis Limitations

Due to the scope of this study, there were several factors that were left unexplored that could help to explain discrepancies in this experiment. Particularly, only one model and dataset were utilized in this experiment, which limits the generalizability of the results. Additionally, sentence-level examples were not provided in this analysis limiting the specific details of how noise affected the accuracy and token ratio on a concrete example. Moreover, noise types were applied independently from each other as well as once per word which does not fully represent real-world noise. Lastly, no statistical analysis was included which does not allow statistically significant or correlation related conclusions to be drawn. Future work could address these issues by incorporating multiple models and datasets, testing multiple combinations of noise types, and including a statistical analysis on the data to more rigorously quantify the results.

6 Mitigation

6.1 Inference-Time Spell Correction

As a mitigation technique, I utilize the "oliverguhr/spelling-correction-english-base" by Guhr et al. (Guhr et al., 2020). The model is a BART-based text-to-text generation model used to fix typographical and punctuation-related errors in text. The BART-based model employs BPE tokenization internally though does not utilize

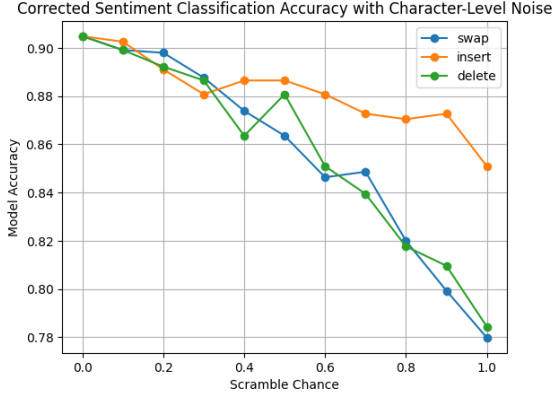


Figure 3: Corrected sentiment classification accuracy of perturbed SST-2 dataset across severity levels for all three noise types.

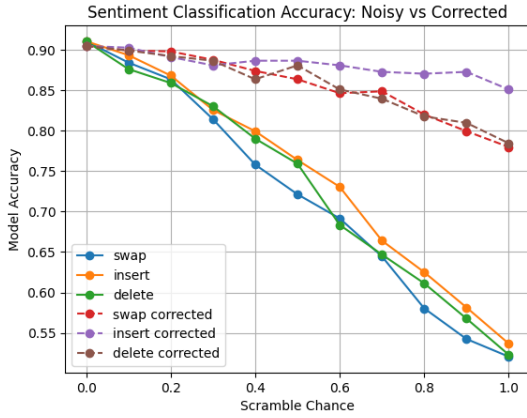


Figure 4: Uncorrected and corrected sentiment classification accuracy of perturbed SST-2 dataset across severity levels for all three noise types.

BPE-Dropout. The noisy text from each severity and noise type is fed into the model and the output is re-evaluated for accuracy. Using this type of spell correction, unlike BPE-Dropout (Provilkov et al., 2020), requires no modification or retraining of the DistilBERT model (Sanh et al., 2019) since spelling correction is already handled at inference-time. This unique benefit allows compatibility with closed-source models where direct access to the internal code is limited and retraining is not possible. Additionally, a model-based spell corrector may be better suited for correcting typographical errors while preserving context compared to algorithmic approaches.

6.2 Recovery Results

As shown in Figure 3, the corrected accuracy is higher at every severity level except 0.0 for all three noise types. I observe that the corrected accuracy does not follow a consistent downward trend across

all severity levels, and each noise type behaves differently under correction, suggesting that the effectiveness of spelling correction is influenced by both noise type and severity level rather than severity alone. First, the corrected accuracy of the insert noise type outperforms the other two noise types at severity levels greater than 0.4 and does not decrease uniformly. Notably, at severity levels 0.4 and 0.5, the corrected accuracy exceeds that of severity level 0.3, contrary to the downward trend observed at other severity levels. Additionally, between severity levels 0.9 and 1.0, a sharp decline in corrected accuracy is observed, suggesting that at the highest severity levels, the degree of perturbation may exceed the corrector’s ability to reliably restore the original text. Second, the corrected accuracy of the delete noise type follows a roughly monotonically decreasing, concave curve. However, a sharp increase in accuracy is observed at severity level 0.5, followed by a sharp decline at severity level 0.6 which returns the curve to its prior trajectory. Third, the swap noise type similarly experiences an increase in corrected accuracy at severity level 0.7. These irregularities across all three noise types suggest that the spelling correction model may be capable of recovering a disproportionately large amount of accuracy when the perturbed input retains enough context for the corrector to operate effectively. Finally, at severity level 1.0, the insert noise type has a corrected accuracy 6.65% higher than the next highest noise type, indicating that insert noise achieves the greatest improvement under correction, particularly at severity levels greater than 0.4. Comparing these corrected results to the uncorrected accuracy in Figure 4 and Table 2, I observe an improvement in accuracy at all severity levels greater than 0.0 which grows substantially at higher severity levels. I also observe that the insert noise type retains the highest uncorrected accuracy among the three noise types and also achieves the greatest corrected accuracy recovery, especially beyond severity level 0.4. These results demonstrate that applying spelling correction at inference-time produces a meaningful improvement in sentiment classification accuracy on perturbed text, which widens substantially as severity approaches 1.0.

6.3 Recovery Limitations

Although the mitigation technique provides a meaningful improvement in the accuracy of the previously perturbed text, the model-based system

Severity	Swap			Insert			Delete		
	Accuracy	TFR	Corrected	Accuracy	TFR	Corrected	Accuracy	TFR	Corrected
0.0	91.06%	1.000	90.48%	91.06%	1.000	90.48%	91.06%	1.000	90.48%
0.1	88.42%	1.067	89.91%	89.33%	1.085	90.25%	87.61%	1.038	89.91%
0.2	86.35%	1.142	89.79%	86.81%	1.172	89.11%	85.89%	1.079	89.22%
0.3	81.42%	1.212	88.76%	82.57%	1.253	88.07%	83.03%	1.118	88.65%
0.4	75.80%	1.284	87.39%	79.93%	1.339	88.65%	79.01%	1.160	86.35%
0.5	72.13%	1.361	86.35%	76.38%	1.417	88.65%	75.92%	1.202	88.07%
0.6	69.15%	1.421	84.63%	73.05%	1.518	88.07%	68.35%	1.246	85.09%
0.7	64.45%	1.497	84.86%	66.40%	1.588	87.27%	64.68%	1.275	83.94%
0.8	58.03%	1.561	82.00%	62.50%	1.681	87.04%	61.12%	1.322	81.77%
0.9	54.24%	1.634	79.93%	58.14%	1.761	87.27%	56.77%	1.360	80.96%
1.0	52.06%	1.702	77.98%	53.67%	1.844	85.09%	52.29%	1.399	78.44%

Table 2: Sentiment classification accuracy, Token Fragmentation Ratio (TFR), and spell-corrected accuracy across severity levels for all three noise types.

requires a considerable amount of computational overhead. Another limitation of the model-based system is the corrected accuracy of the baseline dataset. The corrected accuracy of the baseline was 0.58 percentage points below the clean baseline accuracy of 91.06%. This indicates that although the spelling correction model is effective on the perturbed text, it may introduce minor errors in the clean text, marginally reducing accuracy. While I did not use BPE-Dropout (Provilkov et al., 2020) in my experiment, BPE-Dropout is a well-established training-time alternative for improving tokenization robustness in BPE-based models and would eliminate the need for a separate inference-time correction model. As a result, future work comparing BPE-Dropout against the spelling correction model could help to quantify the improvements in accuracy of each method. Comparison of the TFR of each method could also be used to determine if token fragmentation is consistent with decline in accuracy.

7 Conclusion

In this paper, I examined the robustness of a language model to text containing character-level perturbations. I tested the accuracy and token fragmentation of the model on three different noise types at eleven severity levels. The results demonstrate that input perturbations negatively affect sentiment classification accuracy while increasing token fragmentation across all noise types and severity levels tested. Additionally, the results show that an inference-time spelling correction model substantially improves the accuracy of text perturbed with all three noise types. Future work in this area could include testing a training-time method such as BPE-

Dropout against an inference-time method such as the spelling correction model.

References

- Yekun Chai, Yewei Fang, Qiwei Peng, and Xuhong Li. 2024. [Tokenization falling short: On subword robustness in large language models](#). In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 1582–1599. Association for Computational Linguistics.
- Esther Gan, Yiran Zhao, Liying Cheng, Mao Yancan, Anirudh Goyal, Kenji Kawaguchi, Min-Yen Kan, and Michael Shieh. 2024. [Reasoning robustness of LLMs to adversarial typographical errors](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 10449–10459. Association for Computational Linguistics.
- Oliver Guhr, Anne-Kathrin Schumann, Frank Bahrmann, and Hans Joachim Böhme. 2020. [Training a sequence to sequence model for spelling correction](#). Hugging Face model repository.
- Takeshi Kojima, Yutaka Matsuo, and Yusuke Iwasawa. 2025. [Continual pre-training on character-level noisy texts makes decoder-based language models robust few-shot learners](#). *Transactions of the Association for Computational Linguistics*, 13:831–847.
- Milad Moradi and Matthias Samwald. 2021. [Evaluating the robustness of neural language models to input perturbations](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 1558–1570. Association for Computational Linguistics.
- Sachin Pawar, Manoj Apte, Kshitij Jadhav, Girish Keshav Palshikar, and Nitin Ramrakhiyani. 2025. [Broken words, broken performance: Effect of tokenization on performance of LLMs](#). In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational*

Linguistics, pages 372–385. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. 2020. [BPE-dropout: Simple and effective subword regularization](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1882–1892. Association for Computational Linguistics.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. [DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter](#). ArXiv preprint arXiv:1910.01108.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. [Neural machine translation of rare words with subword units](#). In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. [Recursive deep models for semantic compositionality over a sentiment treebank](#). In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1631–1642. Association for Computational Linguistics.

Dixuan Wang, Yanda Li, Junyuan Jiang, Zepeng Ding, Ziqin Luo, Guochao Jiang, Jiaqing Liang, and Deqing Yang. 2024. [Tokenization matters! Degrading large language models through challenging their tokenization](#). ArXiv preprint arXiv:2405.17067.

Wenxuan Zhang, Yue Deng, Bing Liu, Sinno Pan, and Lidong Bing. 2024. [Sentiment analysis in the era of large language models: A reality check](#). In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 3881–3906. Association for Computational Linguistics.